Repairing Document Knowledge Graphs with Source Text and Field Checks

Hua Peng[1†], Zifu Tao[2†], Yu Guo[1], Qiang Zhou[2], Tian Zhou[3*], Bo Han[3]

[1]China Mobile System Integration Co. Ltd., China.
[2]Tongji University, China.
[3]Xi'an Jiaotong University, China.

*Corresponding author(s). E-mail(s): tianzhou@xjtu.edu.cn;
Contributing authors: penghua@cmict.chinamobile.com;
2253323@tongji.edu.cn; guoyu1@cmict.chinamobile.com;
zhouqiang@cmict.chinamobile.com; bohan@xjtu.edu.cn;
†These authors contributed equally to this work.

Abstract

Knowledge graphs built from text often contain missing facts, wrong values, and repeated triples. Repair needs checks that find these errors, constraints that guide changes, and a way to choose useful edits. We present a framework that links multi-scale quality assessment, constraint-driven repair, and scale-aware routing. Local, graph, and source checks form a shared assessment profile. Source and field constraints guide graph generation and filter the proposed triples. A neural router uses the profile to weight candidate edits. The selector checks each edit on a graph copy and compares its quality gain and cost under graph bounds. It updates the profile after each accepted change. Tests cover Chinese government, finance, and environment records, receipts, and contracts, with component tests and human review. The main pipeline repairs 98% of added defects. On receipts, source-line hints raise F1 over Simple Pipeline by 4.76 points. The evaluation tracks recovered facts and correct facts kept, showing how each repair stage affects the output graph.

Keywords: Knowledge Graph Repair, Source Text, Field Checks, Document Extraction

1 Introduction

Knowledge graphs (KGs) connect facts across sources and support question answering, recommendation, and data analysis (Ji et al. 2022; Pan et al. 2023). These applications depend on the quality of the stored facts. A missing relation can hide useful information, while a wrong value can lead to a wrong answer or decision. As more graphs are built from text, checking and improving their quality has become an important part of KG construction (Zaveri et al. 2016; Paulheim 2017). The goal is to recover useful facts and preserve the correct information already in the graph.

Text-based extraction leaves several kinds of errors in the same graph. Source noise and unclear wording can cause missing fields, repeated facts, wrong relations, and values assigned to the wrong fields (Ji et al. 2022; Paulheim 2017). These errors connect graph structure with the meaning of the source text. For example, both a subtotal and a final amount may appear on a receipt, but they serve different roles. Checking whether an amount appears in the text is one part of deciding whether it belongs in the total field. Graph repair therefore needs both structural checks and evidence for each field value.

Existing methods provide several ways to improve graph quality. Rule-based checks find violations of graph and field requirements (Kontokostas et al. 2014; Knublauch and Kontokostas 2017). Embedding models score how likely a fact is from patterns in the graph (Bordes et al. 2013; Nickel et al. 2016). Large language models (LLMs) use text and graph context to suggest repairs (Pan et al. 2023; Lin et al. 2025). Each approach provides useful information for a repair decision. A rule violation points to a broken requirement. A model proposal offers a possible replacement. Connecting these signals requires a common view of the errors, clear bounds on changes, and a way to choose among repair actions.

Quality errors arise at different scales. Local checks can find repeated facts, while graph checks test relations and conflicts across fields. Source checks connect stored values to document evidence (Zaveri et al. 2016; Wang et al. 2021). A value may satisfy a field rule yet refer to the wrong part of a document. Repair needs a view that shows where an error occurs and what evidence is needed to fix it.

Repair also involves a trade-off between recovering missing facts and preserving correct ones. Adding a fact can create a field conflict, while deleting a fact can remove useful information. A change can also affect other parts of the graph. Removing an edge, for example, may leave a valid node without a connection. Repair therefore needs clear bounds on changes and a cost for each edit.

The choice of repair action must reflect the type of error. Simple rules can remove repeated facts and correct clear structural errors. Missing or wrong values require reading the source text. Several proposed edits may target the same field or change the effect of another edit. Their order matters because an accepted change can alter what needs repair next. The choice should use the current errors, expected gains, and edit costs.

Our idea is to link these decisions through a shared assessment profile. The profile records the errors and quality scores, constraints set bounds on changes, and edit selection uses both to guide repair. Together, they connect the kind of error with

the scope and cost of its repair. We build a framework around this link, combining multi-scale assessment, constraint-driven repair, and scale-aware routing.

We support this idea with *Multi-Scale Quality Assessment* across entity, graph, and context scales. Entity checks find local errors, graph checks test field and relation rules, and context checks use the original document. The checks form a shared profile that records errors and quality scores at each scope. An error report guides model proposals in the main path. The edit selector uses quality scores to compare trial changes. This connects the detected problems with the repair process.

Constraint-Driven Optimization gives the changes clear bounds. The main repair path combines rule-based cleanup, LLM generation, and triple filtering. Source and field requirements define which generated triples can enter the output. Field meanings and source-line hints guide the model toward useful evidence. For edit selection, quality lower bounds and a density limit define allowed trial graphs, while edit costs favor smaller changes. These requirements connect the repair goal with the checks used to accept a proposed change.

Scale-Aware Routing supports a path that chooses edits one at a time. A small neural model uses the profile to weight edits at the entity, graph, and context scales. The selector tests proposed edits on graph copies and compares their quality gains and costs under graph bounds. After an accepted edit, it checks the graph again and updates the profile. This lets the next decision use the changed graph and the errors that remain.

The paper makes three contributions:

- **Multi-scale quality assessment.** We combine local structure, graph rules, and source checks in a shared profile that connects detected defects with repair decisions.
- **Constraint-driven repair and scale-aware routing.** We define source and field constraints for generated graphs. A neural routing path weighs candidate edits and compares their quality gains and costs under graph bounds.
- **Evaluation on document graphs.** We compare repair methods and their components on added defects, model outputs, receipts, and contracts, and include human review. Saved outputs and decision logs support checks of each repair stage.

Experiments show that the main pipeline repairs 98% of added defects in Chinese records and that source-line hints raise receipt F1 over Simple Pipeline by 4.76 points.

2 Related Work

2.1 Building Graphs and Checking Extracted Facts

Graph construction turns source data into linked facts (Ji et al. 2022). DBpedia, YAGO, and Freebase build knowledge bases from structured and partly structured sources (Auer et al. 2007; Suchanek et al. 2007; Bollacker et al. 2008). Quality checks help select facts during this process. Knowledge fusion combines evidence from several sources to score candidate facts (Dong et al. 2014). This connects extraction with the question of which facts to keep.

Learning methods use graph patterns to predict links (Bordes et al. 2013; Wang et al. 2017; Schlichtkrull et al. 2018). Surveys describe how LLMs use text to build,

complete, and combine KGs (Pan et al. 2023, 2024; Bian 2025). Reasoning prompts and prompt patterns guide model responses (Wei et al. 2022; White et al. 2023). Retrieval-augmented generation adds source text to the model input (Lewis et al. 2020). Neo4j Labs' LLM Knowledge Graph Builder applies such models to documents with user-set schemas (Neo4j Labs 2024). These studies provide ways to produce candidate facts. Our task starts from an extracted graph and uses the source document to check proposed field values.

2.2 Graph Quality and Check Scopes

Quality assessment asks whether stored facts are correct, complete, consistent, and current (Zaveri et al. 2016; Paulheim 2017; Wang et al. 2021). One route checks how well the data fit observed patterns. Statistical methods find unusual values from their distributions (Wienand and Paulheim 2014; Paulheim and Bizer 2014). Embedding scores measure how well triples fit learned graph patterns (Nickel et al. 2016). Another route states the requirements for valid records through tests and graph constraints (Kontokostas et al. 2014; Knublauch and Kontokostas 2017). Human review helps assess cases with unclear answers (Acosta et al. 2013).

These approaches turn quality goals into checks that locate different kinds of errors. For document graphs, local structure, field rules, and source evidence provide linked views of the same record. Our assessment profile brings repeated facts, field violations, and source mismatches into one report. This report connects the detected problems with repair choices.

2.3 Repairing Graphs and Choosing Actions

Repair involves recovering missing facts and preserving correct information. Completion methods predict missing links from paths and embeddings (Lao et al. 2011; Bordes et al. 2013; Yang et al. 2015; Shi and Weninger 2017). Neural models also learn scores for possible links (Dettmers et al. 2018; Zhu et al. 2021). Path-based reinforcement learning supports query answering (Das et al. 2018), while few-shot learning supports link prediction from few examples (Chen et al. 2019). These methods provide candidate facts for graph completion.

Rules give proposed facts a structure that can be checked. Rule mining finds positive and negative patterns in knowledge bases (Galárraga et al. 2013; Ortona et al. 2018). Embedding training can include logical rules and simple constraints (Guo et al. 2016; Ding et al. 2018). RNNLogic and pLogicNet combine logical rules with learned predictions (Qu et al. 2021; Qu and Tang 2019). These studies connect fact prediction with rules that guide learning and checking.

Language models provide another route to repair through prompts and context. Lin et al. use violated SHACL constraints and graph context in repair prompts (Lin et al. 2025). ReAct links model reasoning with actions that gather information (Yao et al. 2023). We draw on these designs for the SHACL-context and ReAct-style baselines. Building on these studies, we connect source evidence with field constraints and edit choice. The main path filters proposed triples, while a scale-aware selector compares trial edits by quality gain and cost under graph bounds.

3 Task and Method

A repair method must connect the errors it finds with the changes it makes. A repeated fact calls for a local change. A wrong field value calls for reading the source. Each change must also meet the graph's rules. We use an assessment profile to link these decisions. It records problems at the entity, graph, and context scales. Constraints then set the allowed outputs, while repair decisions use the reported errors and the source text.

3.1 Task and Overall Design

We study document graphs whose relations represent fields. The input is a list of triples G, source text X, a fixed document ID h_0, and allowed relations $\mathcal{R}_X$. The list can contain copies of the same triple. The target graph allows at most one nonempty value per relation; fields are optional. Document records and receipts follow this form. Repair produces a graph $\widehat{G}$ from these inputs. The scorer compares it with the reference graph G^*.

The framework has two repair paths with separate evaluations. The main path uses an error report to guide a model that returns a complete graph. A filter checks the generated triples against source and field rules. The edit-selection path uses quality scores to compare changes one at a time. A router weighs their scopes, and trial checks measure their gains and costs. Both paths link assessment to repair: the report guides value generation, and the scores guide edit selection.

3.2 Multi-Scale Quality Assessment

Errors differ in the information needed to detect them. A repeated triple can be found within the graph, while a copied value must be checked against the document. We therefore group checks by scope. Entity checks cover local links and repeated facts. Graph checks cover document IDs, field names, and relation rules. Context checks compare values with the source text. Here, context is the source document. These groups form an assessment profile that identifies the kind and scope of each problem. Figure 1 shows the groups.

3.2.1 Field Checks and Error Report

Cleanup first fixes errors with clear rule-based changes. The preprocessor P removes duplicate triples. It reverses an edge if its object is h_0 and its subject is a different node. It then removes triples with an invalid head or relation. Here, the head is the subject of a triple. The cleaned graph is $G_s = P(G; h_0, \mathcal{R}_X)$.

The field checks keep structural and source errors separate. For a triple $t = (h, r, o)$, they are

$$H(t) = \mathbf{1}[h = h_0], \quad R(t) = \mathbf{1}[r \in \mathcal{R}_X], \quad S(t; X) = \mathbf{1}[o \neq \emptyset \ \wedge \ o \text{ occurs in } X]. \quad (1)$$

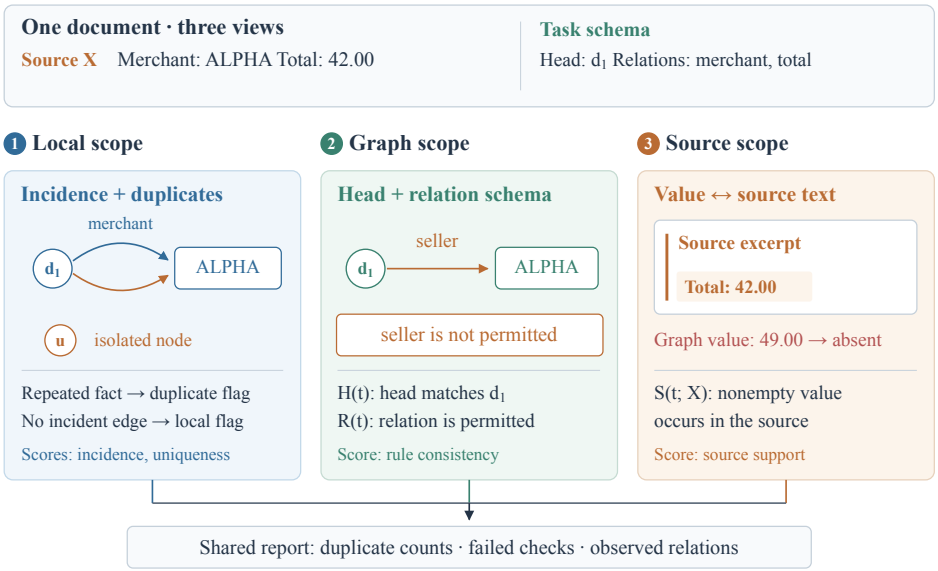


Fig. 1 Three scopes of graph checks. Local checks find repeated facts and unlinked nodes. Graph checks test document IDs and field names. Source checks look for values in the text. Both repair paths use these groups to build an assessment profile.

H checks the document ID, R checks the relation, and S checks the source. A value can pass the first two checks and fail the third. This separation gives a clear reason for each rejected triple.

The main path collects these checks in an error report:

$$D(G_s, X) = \big(n_{\text{dup}}, I_H, I_R, I_S, \mathcal{R}(G_s)\big). \tag{2}$$

The report stores duplicate counts, failed-check positions, and present relations. For example, $I_S = \{i : S(t_i; X) = 0\}$ lists positions that fail the source check. It is built from the cleaned input. Failed positions point to values that need review, while present relations show the fields already in the graph. The model receives this report along with the graph and source text. This gives the generation step an explicit account of the detected problems.

3.2.2 Quality Scores for Trial Graphs

Choosing an edit also requires comparing the graph before and after a change. We use four scores for this purpose: linked nodes, unique triples, rule agreement, and source support.

$$\begin{aligned} \mathbf{s} &= (Q_{\text{inc}}, Q_{\text{uniq}}, Q_{\text{rule}}, Q_{\text{src}}), \\ Q_i &= 100(1 - r_i), \quad Q = \tfrac{1}{4} \sum_i Q_i. \end{aligned} \tag{3}$$

Each r_i is an error rate for its check. For a nonempty graph, the rates are

$$r_{\text{inc}} = \frac{I}{n}, \quad r_{\text{uniq}} = \frac{m - c}{m}, \quad r_{\text{rule}} = \frac{B}{m}, \quad r_{\text{src}} = \frac{V}{m}. \tag{4}$$

I counts unlinked nodes, and c counts triple similarity groups. B counts distinct triples flagged by graph rules. V counts values absent from the text after removing whitespace. Here, n and m count nodes and triples. The four scores use the same scale, and their mean gives the trial quality. The selector retains each score as well, so it can check whether an edit raises one score at the cost of another.

Missing fields need a further check because they have no triple to inspect. The profile therefore also reports fields found in the source but absent from the graph. These reports enter the violation count and candidate list. Together, the field report and quality scores describe where a repair is needed and how a trial change affects the graph.

3.3 Constraint-Driven Graph Repair

A generated graph must choose field values and meet the task's rules. We assign these tasks to generation and filtering. The model uses the source and error report to propose values. The filter then applies fixed source and field checks to the proposal. This design uses the same requirements to describe input errors and to decide which output triples can be kept.

3.3.1 Generating a Graph from Source Text

Rule-based cleanup handles clear structural errors. Recovering a missing value or replacing a wrong one requires reading the source. The language model performs this step and returns an ordered list of triples:

$$C = L_\theta(X, h_0, \mathcal{R}_X, G_s, D(G_s, X)). \tag{5}$$

The prompt asks for a complete graph. It tells the model to copy source values, keep correct fields, and fill missing or wrong values. Returning the whole graph lets the model consider the fields together. The output order also provides a fixed way to choose among competing values during filtering.

3.3.2 Field Meanings and Source Lines

Source text can contain several values that fit the same field format. A receipt may list a brand and a company, or a subtotal and a final amount. We give each field a short meaning so that the model can choose the value that fits its role. All receipt repair methods receive these meanings and numbered source lines.

The source-line index draws attention to likely evidence for each field. Fixed patterns find company terms, address terms, postcodes, dates, and total labels. Each matching line is an anchor. The index includes that line, one line before it, and two lines after it. These nearby lines provide context when a label and its value appear

on different lines. Simple receives the full source and field meanings. Evidence Index also receives the line hints, with the full source still available.

3.3.3 Checking the Generated Triples

The filter turns the task rules into output requirements. A triple must use the given document ID and an allowed relation, and its value must occur in the source. Each field can keep at most one value. Let $\mathrm{uniq}(C)$ remove copies in output order. The triples that pass the checks are

$$C_X = \{t \in \mathrm{uniq}(C) : H(t)R(t)S(t;X) = 1\}. \tag{6}$$

The allowed output sets are

$$\mathcal{F}(C_X) = \left\{ A \subseteq C_X : \sum_{t \in A} \mathbf{1}[r_t = r] \le 1,\ \forall r \in \mathcal{R}_X \right\}. \tag{7}$$

Equation 6 sets the evidence and schema requirements. Equation 7 sets the field-count limit. The first step removes triples that fail a check; the second resolves competition between values assigned to the same field.

The filter scans candidates in order and keeps the first valid value per field. The output $\widehat{G}$ has size

$$|\widehat{G}| = \max_{A \in \mathcal{F}(C_X)} |A| = |\{r : \exists t \in C_X,\ r_t = r\}|. \tag{8}$$

Each field forms one group with room for one triple. Keeping one candidate from every group with a valid value gives the largest allowed output size. Output order breaks ties within a field. Thus, generation selects and orders the values, while filtering enforces source match, schema rules, and the single-value requirement.

The receipt filter uses the same whitespace rule as the parser. $N(s)$ changes each run of spaces or line breaks to one space and trims the ends. The source check is

$$S_{\mathrm{ws}}(t;X) = \mathbf{1}[|N(o)| > 0 \ \wedge\ N(o) \text{ occurs in } N(X)]. \tag{9}$$

It keeps word boundaries, case, and punctuation. The receipt follow-up uses this check. Earlier tests use the original exact-text check.

Algorithm 1 brings these steps together as Diagnosis + Gate. Diagnosis is the error report, and Gate is the triple filter. The main path makes one model call per document. The filter returns both the repaired graph and a log of rejected triples, so each removal can be traced to a check.

3.4 Scale-Aware Routing and Edit Actions

An edit can change what should be repaired next. Several candidates may target the same field, and one change may remove the need for another. The edit-selection path therefore checks the graph again after each accepted change. Scale-aware routing uses

Algorithm 1 Repair with source text and field checks

Require: Input triples G, source X, document ID h_0, allowed relations $\mathcal{R}_X$
Ensure: Repaired graph $\widehat{G}$ and rejection log J

$G_s \leftarrow P(G; h_0, \mathcal{R}_X)$
$D \leftarrow D(G_s, X)$
$C \leftarrow \textsc{Parse}(L_\theta(X, h_0, \mathcal{R}_X, G_s, D))$
$\widehat{G} \leftarrow \emptyset$; $J \leftarrow \emptyset$; $B \leftarrow \emptyset$
for each candidate $t = (h, r, o)$ in generation order **do**
    if t appeared earlier in C **then**
        append $(t, \text{duplicate})$ to J
    else if $H(t)R(t)S(t; X) = 1$ and $r \notin B$ **then**
        $\widehat{G} \leftarrow \widehat{G} \cup \{t\}$; $B \leftarrow B \cup \{r\}$
    else
        append $(t, \text{failed check})$ to J
    end if
end for
return $\widehat{G}, J$

the current assessment profile to weigh local, graph, and source-based edits. Trial checks then compare their gains and costs before selecting an action. Figure 2 shows this path.

3.4.1 Routing from the Assessment Profile

The router needs both the error scores and the size of the graph being repaired. It receives the four quality scores and four graph counts:

$$\mathbf{g} = (n, m, \rho, n_{\text{viol}}), \qquad \rho = \frac{|E_{\text{pair}}|}{n(n-1)} \quad (n > 1). \tag{10}$$

Here, n and m count nodes and triples. E_{pair} contains distinct directed node pairs without self-links. Density is zero when $n \leq 1$. n_{viol} counts the reported violations. These counts describe graph size and reported problems alongside the rates in $\mathbf{s}$. The field report D gives the main path error locations; the selector uses this numerical profile to weigh edit scopes.

The router maps these eight inputs to a repair score and scope weights:

$$(p_{\text{repair}}, \boldsymbol{\pi}) = f_\phi(\text{scale}([\mathbf{s}; \mathbf{g}])). \tag{11}$$

scale uses training-set means and standard deviations. $\boldsymbol{\pi}$ has three nonnegative weights that sum to one. They weigh entity, graph, and context edits. The repair score helps decide when to start trial checks. The scope weights enter the candidate score, linking the current error profile to the choice of the next edit.

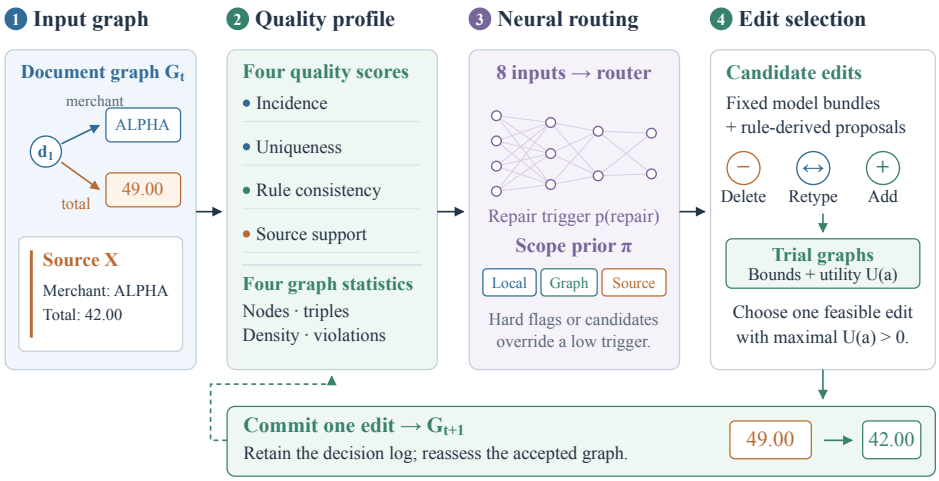


Fig. 2 Scale-aware routing for edit selection. The assessment profile and graph counts feed the router. It gives a repair score and weights for three edit scopes. The selector tries model and rule edits on graph copies, applies the best allowed positive-score edit, and checks again.

3.4.2 Candidate Edits and Costs

Rules and model proposals provide candidate edits. The selector can delete a triple, change its relation, or add a triple. The relation-change operation is called retype in the code. Related edits can form one bundle and be tested together. This lets the selector assess the combined change when several operations affect the same field.

Edit costs give the trial score a preference for smaller changes. Deleting an existing triple has a higher cost than changing its relation or adding a triple. The costs are

$$c(a) = \begin{cases} 0.30 & \text{delete}, \\ 0.16 & \text{retype}, \\ 0.06 & \text{add}. \end{cases} \tag{12}$$

A bundle's cost is the sum of its edit costs. A candidate must earn enough gain in the trial score to offset this cost. This makes the amount and type of change part of the repair decision.

3.5 Trial Checks and Repair Loop

The effect of an edit depends on the graph it changes. We therefore try each candidate on a graph copy and recompute the full profile. Let G_a be the trial graph, h the

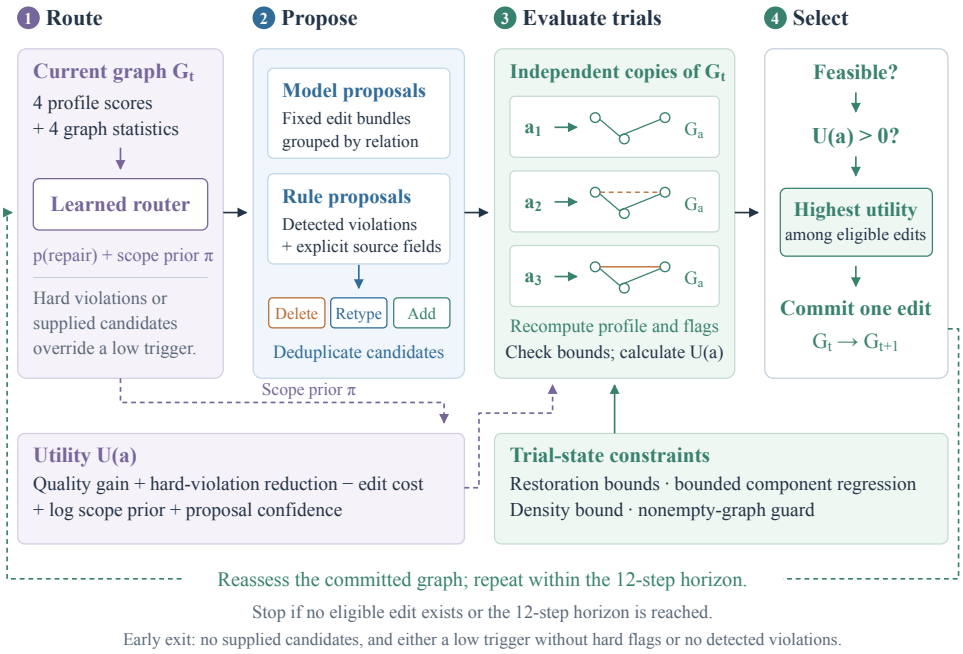


Fig. 3 One round of trial checking. The router weighs edit scopes. The selector scores each model or rule edit on a graph copy, checks the bounds, and applies the best allowed positive-score edit. It then rebuilds the assessment profile. The saved-proposal test allows twelve rounds.

hard-rule error count, $\sigma(a)$ the edit scope, and $z(a)$ its confidence. The default score is

$$U(a) = \frac{Q(G_a) - Q(G)}{100} + 0.20\frac{\max(0, h(G) - h(G_a))}{\max(1, h(G))} - 0.35c(a) + 0.05\log\big(\max(\pi_{\sigma(a)}, 0.05) + 10^{-6}\big) + 0.02z(a). \tag{13}$$

The first two terms reward higher quality and fewer rule errors. The remaining terms account for edit cost, scope weight, and candidate confidence. Together, they compare a proposed change with the current graph and express its gain and cost in one score.

A rise in mean quality can come with a drop in one of its scores. We therefore check each score against a lower bound as well as limiting its decrease. A bound that is already met must remain met. A score below its bound must stay the same or rise. We also limit graph density and require a nonempty graph to stay nonempty. These requirements define the allowed trial set $\mathcal{A}_t^{\mathrm{ok}}$. The selection step is

$$a_t^* = \arg\max_{a \in \mathcal{A}_t^{\mathrm{ok}}} U(a), \qquad U(a_t^*) > 0. \tag{14}$$

Equation 14 chooses among the current trial edits. The bounds decide which edits are allowed; the score orders the allowed edits. The graph stays unchanged when no allowed edit has a positive score.

Algorithm 2 Scale-aware edit selection with trial checks

Require: Input graph G, source X, supplied candidates A_0, task inputs, step limit T
Ensure: Repaired graph $\widehat{G}$ and decision log J

$\widehat{G} \leftarrow G$; $J \leftarrow \emptyset$; $E \leftarrow \emptyset$
add missing source-field candidates to A_0
for $t = 1$ to T **do**
    compute profile and $(p_{\text{repair}}, \boldsymbol{\pi})$ for $\widehat{G}$
    if $A_0 = \emptyset$, $h(\widehat{G}) = 0$, and $p_{\text{repair}} < \tau_{\text{repair}}$ **then**
        break
    end if
    if no reported violations and $A_0 = \emptyset$ **then**
        break
    end if
    $A \leftarrow \textsc{Unique}(A_0 \cup \textsc{RuleEdits}(\widehat{G}))$
    skip graph–candidate pairs already in E
    try each remaining $a \in A$ on a copy and record its pair in E
    check graph bounds and compute $U(a)$ for each trial
    append trial scores and decisions to J
    if no allowed candidate has $U(a) > 0$ **then**
        break
    end if
    apply the allowed edit with the highest $U(a)$ to $\widehat{G}$
end for
return $\widehat{G}, J$

Algorithm 2 links assessment, routing, and execution. After an accepted edit, the next round uses the changed graph and a new assessment profile. Supplied candidates or hard-rule errors start trial checks even with a low repair score. The loop ends when no useful allowed edit remains, no trial is needed, or the step limit is reached. Logs save trial graphs, scores, decisions, and stop reasons.

4 Implementation

The following settings specify how the assessment, generation, and edit selection steps are run. We give the scoring rules, router training, trial limits, and prompt comparisons, followed by the cost of the checks.

4.1 Assessment Settings

Triple similarity uses character-pair counts for the subject, relation, and value. Their cosine similarities have weights 0.4, 0.2, and 0.4. Pairs at or above 0.92 form groups for the uniqueness score. Hierarchy checks use management, supervision, and membership directions. They skip relations with unclear meanings. Entity checks apply to endpoints marked as entity references. Other field values are treated as text.

An empty graph has node-link score zero and the other three scores 100. For a nonempty graph without source text, the source score is 50. The selector's source check removes whitespace from both strings. Earlier selector logs retain the whitespace rules used in those runs.

The missing-field check reads values after allowed labels at a document, line, or sentence start. It adds a candidate when the text gives a field that the graph omits. It skips a field when repeated labels give conflicting values. This check supplies candidates from explicit source fields.

4.2 Router and Training Settings

The network has layers of size $8 \to 32 \to 16$, one repair output, and three scope outputs: 884 parameters in all. Both hidden layers use ReLU. A sigmoid gives the repair score, and a softmax gives the scope weights. Training and use share the same feature code. A feature-version mismatch switches the router to equal weights.

Training labels come from the known added defects. The repair label y_j is one for a degraded graph and zero for its clean pair. The scope label c_j is the most frequent defect scope in that graph. Clean graphs train the repair output only. The loss is

$$\begin{aligned}\mathcal{L}(\phi) = \frac{1}{N}\sum_{j} \mathrm{BCE}(p_j, y_j) \\ + \frac{\sum_j b_j(-\log \pi_{j,c_j})}{\sum_j b_j + \epsilon}.\end{aligned} \tag{15}$$

b_j is one when a scope label is present, and ϵ avoids division by zero. The first term trains the repair score. The second trains the scope weights on degraded graphs. Document groups stay within one training, validation, or test split. The selector uses task inputs and source text to compute its features.

4.3 Candidate and Trial Settings

In the saved-proposal test, changes between the cleaned input and model output are grouped by relation. Each group is a bundle with confidence 0.7. The learned-prior test compares the router with an always-on equal-weight prior.

The lower bounds are 55, 55, 60, and 45 for the four profile scores. Each checked score can fall by at most two points, subject to the bound rules above. Trial density is at most 0.75. The saved-proposal test uses $T = 12$ and $\tau_{\mathrm{repair}} = 0.05$.

A second score subtracts $0.05 \log(1/3 + 10^{-6})$ from the default score. This sets the prior term to zero for equal weights. It keeps the edit order for a fixed router but shifts the positive-score cutoff. Both scores are fixed before the saved-proposal tests.

4.4 Prompt and Rule Settings

4.4.1 Model Inputs and Output Parsing

The runner passes the source, document ID, allowed relations, and input triples to the model. The scorer loads the reference values. Tests change references and defect labels

and check that model inputs stay the same. The older loader reads the document ID from the reference record as task information.

The JSON parser returns an empty graph for an invalid response. Scoring includes these empty graphs. The base prompt uses the same inputs and output limit as the error-report prompt, with the report removed.

4.4.2 Shared Prompts and Responses

One system prompt supports three settings: basic input, an error report, and SHACL checks. Three calls give three responses for each input. We score each response before and after filtering, giving six scores. Each before/after pair uses the same triples. This keeps candidate generation fixed when measuring the filter.

The model is `google/gemma-4-26B-A4B-it` at temperature zero. The output limit is 4,000 tokens in each setting. We report input tokens because prompt lengths differ. Request times include retries and pacing waits. We save all responses and score invalid responses too. The earlier Claude tests use a separate prompt for each method.

4.4.3 SHACL Prompt Baseline

Lin et al. use five prompt parts: an introduction, detected errors, SHACL rules, graph context, and instructions (Lin et al. 2025). We use their full-rules/full-graph ($M + G$) setting for the field task. The pySHACL tool checks document IDs, allowed relations, nonempty values, and field counts. A SPARQL rule checks values against the source text.

The prompt includes the full source and returns a complete JSON graph. It groups each document's errors into one call. We call this baseline *Lin-style SHACL context (adapted)*. RDF stores unique triples. Our shared JSON input keeps duplicate copies. We score the output before and after filtering for each prompt setting.

4.5 Computational Cost

With hash-based lookups, cleanup takes $O(|G|)$ triple operations on average. Filtering k candidates costs $O(k(1 + c_X))$ and uses $O(k + |\mathcal{R}_X|)$ extra memory. Here, c_X is the cost of finding a value in the text. Text length affects this cost. We time model calls apart from these checks.

Pairwise triple checks cost $O(m^2)$ for m triples. T rounds with up to K trials per round cost $O(TKm^2)$, plus text checks. The main filter scans the candidate list once.

5 Evaluation

The evaluation follows the repair decisions in the method. We first compare complete methods on added defects and model-built graphs. We then test error reports, filtering, and edit selection under shared conditions. Receipt and contract tests examine field choice in source text, and human review checks the meaning of errors and edits. We ask three questions. **RQ1**: when does repair improve a graph? **RQ2**: what do context, filtering, and edit selection change? **RQ3**: when does an existing graph help more than fresh extraction? Table 12 brings the findings together.

Table 1 Test data with added defects. The last three columns count documents.

Domain	Documents	Defects	Train	Val.	Test
Government	499	998	349	75	75
Finance	500	1,000	350	75	75
Environment	500	1,000	350	75	75
Total	1,499	2,998	1,049	225	225

5.1 Test Setup

Source records and added defects.

The source data contain 4,728 government records from the Shaanxi Provincial Government, 1,000 finance records from the National Financial Regulatory Administration, and 500 environment records from the Ministry of Ecology and Environment. We sample up to 500 documents from each source and build a field graph for each one.

The source text lists nonempty fields as "field name: value". Chinese sentence marks join the fields. Each value has at most 320 characters; the service label has at most 100. The same cleaned values form the reference triples. We split documents into train, validation, and test groups at 70/15/15 with seed 42. We split by source ID before adding defects. Each original document and its changed copy belong to the same group.

Each test graph has exactly two added defects. The six types are missing triple, duplicate triple, invalid relation, reversed edge, unsupported value, and hierarchy conflict. The saved data list every clean and changed triple. The test set has 225 documents, 75 per source, and 450 defects (Table 1).

Graphs built by a model.

Claude Haiku builds a new graph for each of the same 225 test documents. It receives the source text, document ID, and allowed relations. The scorer uses the structured fields as reference values. Ten responses have invalid JSON and are scored as empty graphs. Parse success is 95.56%. In total, 77 graphs differ from the reference, with 302 triple differences. Two reviewers check a fixed sample of 200 differences. A third reviewer resolves their disagreements (Section 5.5).

Repair methods.

All methods receive the same input graph, source text, document ID, and allowed relations.

- **Source-field Copy** reads field labels and copies values directly from the text.
- **No Repair** returns the input.
- **Rule Only** removes copies, reverses an edge that ends at the document node, and removes invalid heads and relations.

- **SHACL-style Repair** checks the document head and allowed relations using SHACL shapes (Knublauch and Kontokostas 2017).
- **Direct LLM** makes one call and returns a complete graph based on the source text.
- **ReAct-style Agent** makes one call to find errors and another to act on them (Yao et al. 2023).
- **Simple Pipeline** cleans the input, makes one call, and removes copies from the response.
- **Diagnosis + Gate** adds an error report to the prompt. It filters the output by document ID, relation, source match, duplicate count, and number of values per field.

The model-based methods use `claude-haiku-4-5-20251001` (Anthropic 2025) at temperature zero on the same endpoint, except where another model is named. All scores include failed calls. On the added-defect test, Direct LLM and Simple each have one invalid JSON response. Diagnosis + Gate and ReAct have zero.

Scores and statistical tests.

A defect is repaired when the output restores the count of each clean triple and removes its changed form. We also score correct facts kept, over-repair, triple F1, exact graph match, calls, and request time. Exact graph match checks both the triples and their copy counts.

Let G_0 be the model-built input, G_1 the repaired graph, and G^* the reference. Let d count extra and missing triples, including copies. The reduction in reference differences is

$$1 - \frac{d(G_1, G^*)}{d(G_0, G^*)},$$

averaged over the 77 inputs that differ from the reference. A negative value means the output has more differences. Fact retention and over-repair also use reference triples. Human scores measure whether reviewers find an error and accept an edit.

Confidence intervals (CIs) use 10,000 paired bootstrap samples. For matched tests, each sample selects whole documents. This keeps the two added defects of a document together. Paired sign-randomization tests use 10,000 draws and seed 42. The earlier defect and exact-match comparisons use exact McNemar tests. The model-output difference counts also use a one-sided paired Wilcoxon test.

The first matched-context tests use unadjusted p values. Later component, receipt-context, and saved-response tests use the Holm groups stated with their tables. Each 95% CI describes one comparison. Holm correction applies to the p values within each test group.

Comparison design.

Complete-method tests measure the result of all repair steps together. Prompt tests keep the model, input, and output limit fixed while changing the context. Filter tests score the same generated triples before and after checks. Selector tests reuse saved proposals and change the rule for accepting edits. These comparisons measure generation, filtering, and action choice at their respective stages.

Table 2 Repair on 225 test documents with 450 added defects. Lower over-repair, call count, and request time are better.

Method	Defect repair	Facts kept	Over-repair	Triple F1	Exact match	Calls/doc	Time/doc
Source-field Copy	**1.0000**	**1.0000**	**0.0000**	**1.0000**	**1.0000**	0.00	< 0.001 s
No Repair	0.0000	**1.0000**	**0.0000**	0.7615	0.0000	0.00	< 0.001 s
Rule Only	0.3200	**1.0000**	**0.0000**	0.8472	0.0622	0.00	< 0.001 s
SHACL-style Repair	0.0000	**1.0000**	**0.0000**	0.7998	0.0000	0.00	< 0.001 s
Direct LLM	0.9244	0.9687	0.0758	0.9520	0.8267	1.00	7.25 s
ReAct-style Agent	0.8911	0.9324	0.1667	0.9213	0.5956	2.00	20.50 s
Simple Pipeline	0.9511	0.9723	0.0705	0.9634	0.8400	1.00	9.27 s
Diagnosis + Gate	0.9800	0.9932	0.0167	0.9917	0.9333	1.00	11.44 s

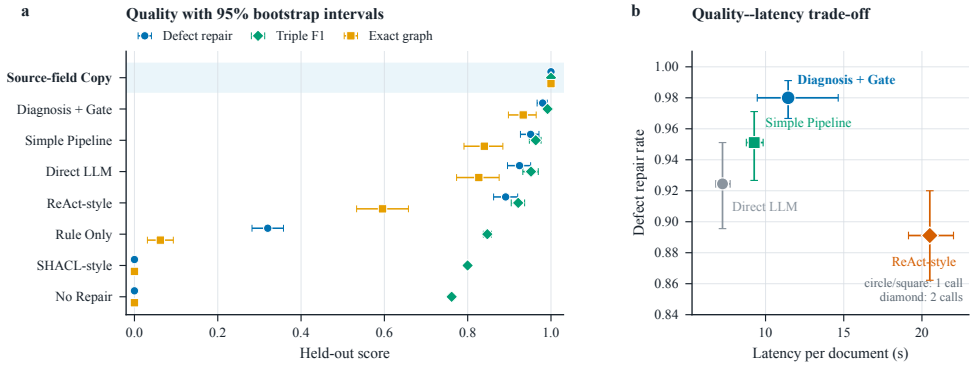


Fig. 4 Repair of added defects. **a**, Repair rate, triple F1, and exact match with 95% CIs from document sampling. **b**, Scores and request times for methods using one or two calls.

5.2 Overall Repair Results

We test whether repair restores missing or wrong facts while keeping correct input facts. Added defects give known repair targets. Model-built graphs test the errors left by extraction. The two-model comparison checks how the complete methods behave with different generators.

5.2.1 Repair of Added Defects

Diagnosis + Gate repairs 98.00% of defects, keeps 99.32% of clean facts, and reaches 99.17% F1 (Table 2). Simple uses the same one-call limit and reaches 95.11% repair and 96.34% F1. Diagnosis + Gate repairs 15 defects that Simple misses. Simple repairs two that Diagnosis + Gate misses (McNemar $p = 0.00235$). The paired repair gain is 2.89 points (95% CI: 1.11–4.89; $p = 0.0067$). The F1 gain is 2.84 points (95% CI: 1.65–4.25). Direct LLM reaches 92.44% repair. Source-field Copy reaches 100% repair and exact match with zero model calls. It reads the field names and values directly from these records.

Diagnosis + Gate repairs 28 defects that Direct LLM misses. Direct LLM repairs three that Diagnosis + Gate misses ($p = 4.65 \times 10^{-6}$). The same counts against ReAct are 42 and two ($p = 1.13 \times 10^{-10}$). These tests compare complete repair methods. Section 5.3 tests their steps.

Table 3 Repair of model-built graphs. Difference reduction uses the 77 inputs that differ from the reference. Other scores use all 225 documents.

Method	Parse	Non-exact inputs	Difference reduction	Facts kept	Over-repair	Triple F1	Exact match
Source-field Copy	1.0000	77	1.0000	1.0000	0.0000	1.0000	1.0000
Extracted graph, no repair	0.9556	77	0.0000	1.0000	0.0000	0.8799	0.6578
Simple Pipeline	1.0000	77	−0.5915	0.9971	0.2389	0.8807	0.6711
Diagnosis + Gate	0.9956	77	0.2775	1.0000	0.0292	0.9248	0.6844

The combined pipeline improves both recovery and fact retention over Simple under the same call limit. The direct-copy result also explains the role of source format: these records state each field name beside its value, so a parser can recover the target graph directly. This makes the test a clear check of recovery from known changes. The receipt and contract tests examine field choice in less regular text.

5.2.2 Repair of Model-Built Graphs

We next repair graphs produced by extraction, using the same source records and references. This tests recovery from model errors and the preservation of fields that were already correct.

The input graphs have 87.99% F1 and 65.78% exact match (Table 3). Diagnosis + Gate raises F1 to 92.48%. It reduces reference differences by 27.75% and keeps all input triples that match the reference. Simple has a mean difference reduction of −59.15% on the 77 inputs with differences. It adds more differences than it removes.

Diagnosis + Gate leads Simple by 4.40 F1 points (95% CI: 3.07–5.77). It leaves 1.03 fewer differences per document on average (95% CI: 0.77–1.32; Wilcoxon $p = 3.18 \times 10^{-12}$). Three graphs match exactly under Diagnosis + Gate and differ under Simple. Zero show the reverse ($p = 0.25$). Source-field Copy matches all references.

Valid and invalid JSON inputs.

The 215 valid JSON inputs have 92.09% F1. Diagnosis + Gate reaches 93.53% and Simple reaches 88.85% on these inputs. Among the 67 valid inputs with reference differences, Diagnosis + Gate reduces the differences by 24.00% and raises F1 from 74.61% to 79.22%. It keeps all 148 initially exact graphs exact. The ten invalid inputs reach 69.91% F1 after repair; zero match exactly.

The valid-input results show repairs within parsed graphs, and the invalid-input group measures recovery from empty graphs. Keeping all initially exact graphs exact shows that the main pipeline preserves those complete inputs in this test. The larger change in F1 than in exact match points to field-level gains: correcting some fields can improve a graph while other differences remain.

5.2.3 Tests with Two Models

We compare Direct LLM, Simple, and Diagnosis + Gate on 60 added-defect inputs, 20 per source. The models are Claude Haiku and `google/gemma-4-26B-A4B-it` (Gemma Team 2026). Each method uses the same inputs, prompt form, temperature, and one-call limit across models. Diagnosis + Gate has the highest repair rate with both models (Table 4). Its F1 lead over Simple is larger with Claude. Gemma's direct outputs have higher F1 than Claude's.

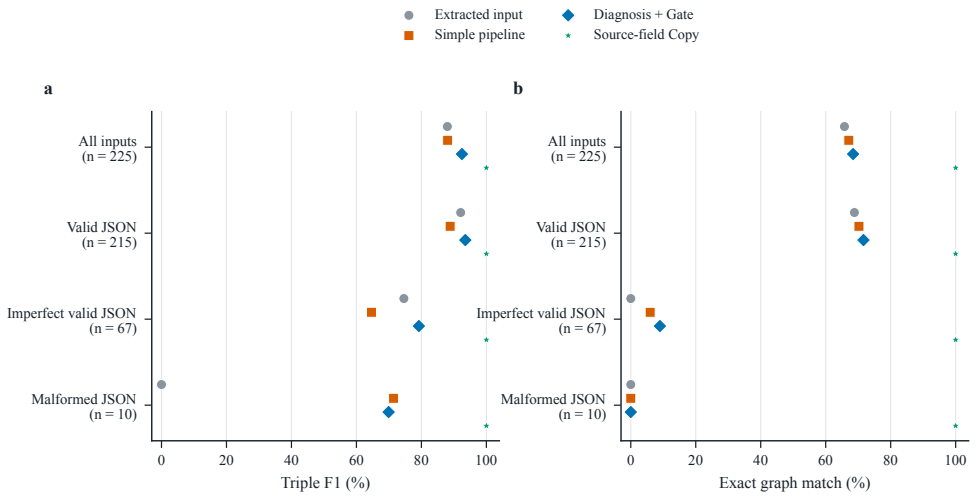


Fig. 5 Repair scores by input type. The valid-JSON group shows repair of parsed graphs. The invalid-JSON group shows recovery from empty inputs.

Table 4 Two-model results on the same 60 added-defect inputs.

Model	Method	Repair	F1	Exact
Claude	Direct	0.958	0.970	0.867
Claude	Simple	0.958	0.966	0.833
Claude	**Diag.+Gate**	**0.975**	**0.991**	**0.917**
Gemma	Direct	0.958	0.988	0.883
Gemma	Simple	0.967	0.984	0.867
Gemma	**Diag.+Gate**	**0.983**	**0.990**	**0.917**

The pipeline has the highest repair rate with both models in this comparison. The size of the F1 gain changes with the generator. This links the complete pipeline's result to the proposals it receives; the shared-prompt and saved-response tests below examine the individual steps.

5.3 Effects of the Repair Components

We now examine how the assessment report, output constraints, and edit selector affect repair. The comparisons follow the stage where each component acts: reports change the prompt, the gate changes generated output, and the selector decides which proposed edits enter the graph.

Table 5 Gemma prompt tests on 60 documents per input group. Each raw/gated pair uses the same response.

Context	Output	Ctrl. repair	Ctrl. F1	Natural F1	Natural exact
Base	Raw	0.9750	0.9951	0.9185	0.6667
Base	Gate	0.9750	0.9962	0.9252	0.6667
Diagnostic	Raw	0.9250	0.9850	0.9079	0.6667
Diagnostic	Gate	0.9250	0.9861	0.9147	0.6667
SHACL context	Raw	0.9500	0.9912	0.9322	0.7000
SHACL context	Gate	0.9500	0.9912	0.9364	0.7000

5.3.1 Error Reports and Prompt Context

The Gemma test uses 60 documents, 20 per source, with added-defect and model-built inputs. Three prompts share the same instructions, cleanup, temperature, and output limit. Each response is scored before and after the same gate. Table 5 gives all six settings. Table 20 gives input tokens and request times. Section 4.4.3 gives the SHACL prompt.

With filtering, the base prompt reaches 99.62% F1 and 97.50% repair on added-defect inputs. The error-report prompt reaches 98.61% F1 and 92.50% repair. The paired F1 change is -1.01 points (95% CI: $[-1.67, -0.38]$; $p = 0.0082$). On model-built inputs, Base + Gate reaches 92.52% F1, Diagnosis + Gate reaches 91.47%, and SHACL Context + Gate reaches 93.64%. The Diagnosis–SHACL F1 difference is -2.16 points (95% CI: $[-5.26, 0.02]$; $p = 0.105$).

The error report lowers mean F1 relative to the base prompt in both input groups. Its list of detected problems changes the model's proposals, but the base prompt produces better field matches in this comparison. SHACL context gives the highest mean F1 on model-built inputs. Thus, the content of the prompt matters alongside the checks applied to its output.

5.3.2 Filtering the Same Generated Triples

On the saved Claude responses, filtering raises F1 from 98.95% to 99.17% and reduces over-repair by 0.60 points. Repair stays at 98.00% and exact match at 93.33%. The filter removes eight triples. All eight fail the source check and differ from the reference. Appendix A.1 gives the full method table.

On model-built inputs with an error-report prompt, filtering raises F1 from 90.79% to 91.47%. The gain is 0.69 points (95% CI: 0.20–1.29; $p = 0.0335$), with the same exact-match score. Across the 360 responses, the gate rejects 27 triples. All 27 differ from the reference.

Tests of each filter check.

We score 540 saved responses under eight filter settings, giving 4,320 outputs. The full filter rejects 28 triples: 11 have wrong document IDs and 17 fail source matching. All 28 differ from the reference. The other checks remove zero extra candidates in this

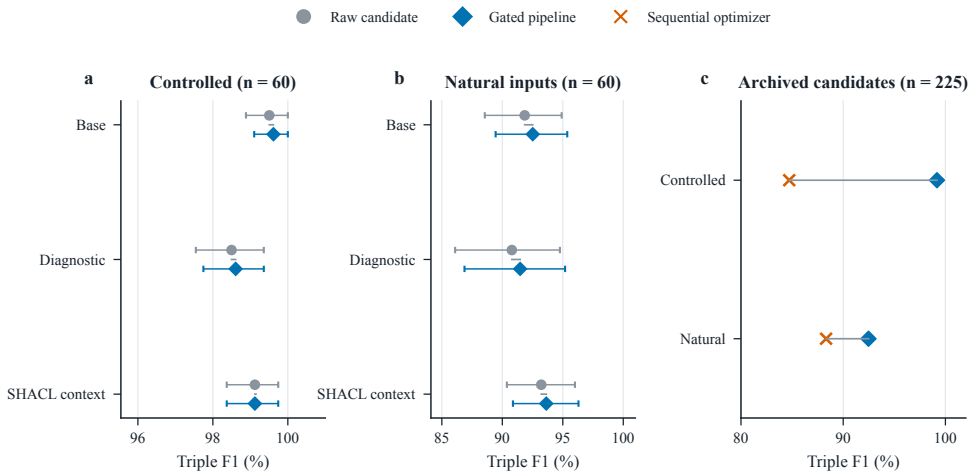


Fig. 6 Prompt, filter, and selector tests. Panels a–b use Gemma outputs on 60 documents. Each raw/gated pair uses the same response. Bars show 95% document-bootstrap CIs. Panel c uses saved Claude proposals for 225 documents.

Table 6 Mean F1 changes in percentage points. Each input group has 60 paired documents. Each effect averages over the other two switches. CIs are 95% document-bootstrap intervals. p_H corrects the six main-effect tests with Holm correction.

Input	Component	Change	95% interval	p_H
Controlled	Preprocessing	-0.028	[-0.797, 0.778]	1.000
Controlled	Diagnosis	-0.651	[-1.348, -0.056]	0.164
Controlled	Gate	+0.164	[0.054, 0.301]	0.156
Natural	Preprocessing	+0.697	[0.000, 1.905]	1.000
Natural	Diagnosis	-0.535	[-2.376, 0.842]	1.000
Natural	Gate	+0.707	[0.225, 1.316]	0.086

sample. All 21 adjusted check-comparison p values exceed 0.05. Appendix A.2 gives the table.

Source and document-ID checks account for the rejected triples in these tests. The gate improves field-level scores by removing candidates that fail those requirements. It keeps the repair and exact-match scores unchanged in the saved Claude comparison. Choosing the right source value for a field still takes place during generation and candidate ordering.

5.3.3 Cleanup, Reports, and Filtering Together

We switch input cleanup (P) and the error report (D) on or off. The four prompt settings use 60 added-defect and 60 model-built inputs, giving 480 Gemma responses. Each response is scored with and without the gate (G). Each main effect averages over the other two switches.

All six Holm-corrected p values exceed 0.05 (Table 6). The error report lowers mean F1 in both input groups. The gate raises mean F1 by 0.164 points on added-defect inputs and 0.707 points on model-built inputs. Across all input groups, the full filter rejects 47 triples: 17 have wrong document IDs and 30 fail source matching. All 47 differ from the reference. The released scores also list the joint switch effects and removals by each check.

Averaging over the other switches gives the gate a positive mean effect and the error report a negative mean effect in both groups. The same directions appear in the matched-context comparison. The rejection logs explain the gate's observed changes through specific source and ID failures; the report acts earlier by changing generation.

5.3.4 Choosing Edits under Graph Bounds

The selector tests ask whether profile scores, edit costs, and scope weights select useful changes from saved proposals. On the saved Claude proposals, the default selector with learned or equal weights accepts zero edits on added-defect inputs. Repair stays at 32.00% and F1 at 84.72%, the cleaned-input scores. On model-built inputs, the current selector reaches 88.33% F1 with equal weights and 87.99% with learned weights. The learned trigger also reaches 87.99%. Removing edit cost raises added-defect F1 to 89.30%. Model-built F1 is 88.13% in that setting.

The logs show missing fields in graphs with high profile scores. Some useful additions receive scores of zero or less and are rejected. Together with the cost test, this points to the acceptance score as a key part of edit selection: an available correction must receive enough gain to pass the positive-score cutoff. Equal scope weights score above learned weights on model-built inputs. Appendix A.2 gives all 13 settings, two prior score forms, and 11,700 outcomes. Appendix A also gives earlier selector results and the call-policy test.

5.4 Repair on Receipts and Contracts

These tests examine fields that must be chosen from document text. The receipt tests compare source-line hints, their contents, and the use of an existing graph. The contract test applies the same repair idea to another document task. We track both recovered fields and correct input matches that are lost during repair.

5.4.1 Source-Line Hints on New Receipts

We use SROIE receipts (Huang et al. 2019) to test whether field-line hints help the model choose values from the full source. The comparison holds field meanings and source text fixed while changing the added context. Appendix A.3 gives the first receipt test and development details.

We set the index design on 20 new receipts, then fix it before testing 60 further receipts (Section 3.3.2). The first 60, development 20, and new test 60 receipts have different IDs and different text after whitespace cleanup. Sampling uses seed 20260923. Test labels are downloaded after all predictions are saved. Every method receives the same field meanings, numbered source lines, input graph, and instructions. Simple uses

Table 7 Repair on 60 new receipts. Methods share field meanings, source lines, and the same whitespace check. Exact match uses the labelled fields.

Method	Triple F1	Exact graph	Normalized F1	Facts kept
Extracted input	0.7833	0.3500	0.7958	1.0000
Simple, raw	0.7875	0.2833	0.8083	0.9250
Simple, gate	0.7875	0.2833	0.8083	0.9250
Evidence Index, raw	0.8333	0.4000	0.8625	0.9306
Evidence Index, gate	0.8351	0.4000	0.8643	0.9306
SHACL context, raw	0.7833	0.2833	0.8042	0.9347
SHACL context, gate	0.7833	0.2833	0.8042	0.9347

them directly. Evidence Index adds field-line hints. SHACL adds rule-check context. All three use the same whitespace rule in the filter.

The main comparison is Evidence Index minus Simple in mean document F1. The code and scoring rules are fixed before testing. Table 7 gives the output scores.

On the new test, input F1 is 78.33%. Simple, Evidence Index, and SHACL reach 78.75%, 83.51%, and 78.33% after filtering. Simple gains 0.42 points over the input (95% CI: $[-4.17, 5.00]$). Evidence Index leads Simple by 4.76 points (95% CI: $[2.50, 7.26]$; paired randomization $p = 0.0004$). Exact match is 28.33%, 40.00%, and 28.33% for the three methods.

The input has 52 fields that differ from the reference. Simple recovers 16 of them and loses 15 exact input matches. Evidence Index recovers 26 and loses 14. SHACL recovers 13 and loses 13. The index fixes 11 punctuation differences and 6 text-boundary differences. Among its 14 lost exact matches, 10 keep the same numeric amount and change the display format.

A numeric check removes currency prefixes and thousands separators from the total field. It gives 58/60 correct amounts for the input, 59/60 for Simple, 59/60 for Evidence Index, and 58/60 for SHACL. After case and whitespace cleanup, F1 is 86.43% for Evidence Index and 80.83% for Simple. The index improves both exact and cleaned-text scores.

The filter rejects 1 candidate. It differs from the reference and joins address parts into text absent from the source. The count of rejected reference matches is 0. Filtering raises indexed F1 by 0.18 points. The other two methods keep the same F1.

The indexed prompt recovers more reference fields than Simple and loses fewer exact input matches. The amount check also explains why some lost exact matches keep the same numeric value. Most of the F1 difference is already present before filtering, placing the main gain at the generation stage. The improvement under case and whitespace cleanup also extends beyond those formatting differences.

5.4.2 Comparing Source-Line Hints

We next test which parts of the indexed prompt affect field choice. Six prompt settings use the same 60 new-test receipts. This separate batch contains 360 new Gemma responses collected with the component tests. All responses parse. Each setting keeps the full source and shares the model, output limit, and filter. The random index has about the same character length as the full index.

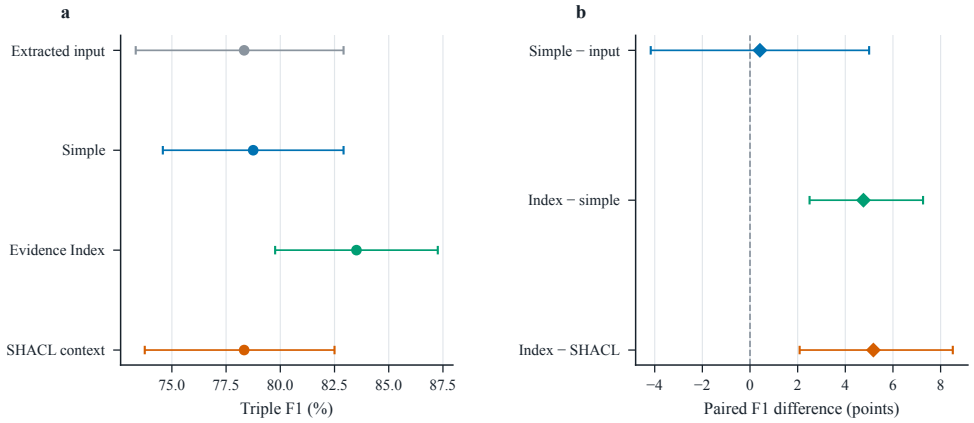


Fig. 7 New receipt test. Left: mean F1 with 95% document-bootstrap CIs. Right: paired F1 changes. All methods share field meanings and source lines.

Table 8 Source-line tests on 60 receipts. Each row uses the same model, source, output limit, and filter. Random hints have about the same character length as the index.

Context	Exact F1	Normalized F1
Simple	0.7917	0.8125
Full index	0.8351	0.8643
Anchors only	0.8351	0.8643
Random index	0.8167	0.8458
Simple, no definitions	0.8083	0.8292
Full index, no definitions	0.7470	0.7762

The full index leads Simple by 4.35 F1 points (95% CI: 1.25 to 7.86). Its Holm-corrected value is $p_H = 0.065$ across five planned comparisons. The full index and anchor lines alone have the same mean F1. The full–random difference is 1.85 points (CI: $[-0.65, 4.52]$; $p_H = 0.696$). Removing field meanings from the full index lowers F1 by 8.81 points (CI: $[4.23, 13.39]$; $p_H = 0.006$). The Simple setting scores higher with field meanings removed in this batch. These results show that field meanings help the indexed prompt choose values.

Anchor lines alone reach the same mean score as the full line windows in this batch. The full–random interval spans gains and losses. Field meanings have a clear role within the indexed prompt, while their effect in Simple goes in the other direction. The useful context therefore depends on how the field description and line hints are presented together.

5.4.3 Repair versus Fresh Extraction

To test the role of an existing graph, we use the 60 new-test receipts in four settings: repair or fresh extraction, with or without the index. All settings share source lines,

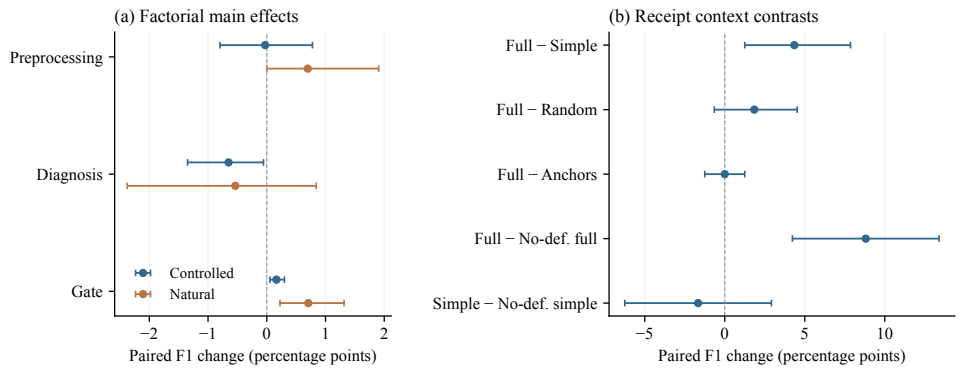


Fig. 8 Component and source-line tests. Points show mean paired F1 changes. Bars show 95% CIs for each change. The component and receipt tests use separate Holm groups.

field meanings, the document ID, and the output limit. Repair also receives the existing graph. All 240 responses contain JSON inside Markdown fences; the fixed strict parser scores them as failures, giving zero F1 in every setting.

In a separate analysis after collection, we rescore these saved responses with the fences removed and the known document ID assigned to every output. Indexed repair reaches 83.93% F1 and indexed fresh extraction reaches 74.86%. The paired difference is 9.07 points (95% CI: 5.22 to 13.10). Under this scoring rule, the existing graph helps produce more matching field values. Its creation cost is counted apart from repair. Appendix A.4 gives the parsing steps, ID errors, and source-line coverage.

5.4.4 Contract Field Test

The contract test examines the same repair idea on a different field task. It compares repair with both its input graph and a fresh extraction, so the result captures gains and losses from changing an existing graph.

We use CUAD (Hendrycks et al. 2021) to test five fields: document name, agreement date, effective date, expiration date or initial term, and governing law. We select contracts with at most one distinct reference value per field after whitespace cleanup and at most 40,000 source characters. We keep the full source and check the positions of the labelled values. Twenty official-training contracts form the development set. All 56 official-test contracts that meet the rules form the test set.

We compare initial extraction, Simple repair, indexed repair, and indexed fresh extraction. Both repair methods receive the same initial graph. The indexed methods add field-line windows to the full source. All settings share the Gemma model, field meanings, and a 4,000-token output limit. Before collection, we set the code to handle JSON fences and assign document IDs. We keep the prompts and selection rules fixed after one development round.

Indexed repair reaches 44.84% F1, compared with 46.76% for the input, a 1.92-point drop. It removes five correct input facts and adds twelve facts that differ from the reference. Simple repair also scores below the input.

Table 9 Contract results on 56 documents. F1 checks exact text after whitespace cleanup. Lost counts correct input facts removed. The input row gives the initial graph cost; repair rows give the repair-call cost.

Output	F1 (%)	Lost	New incorrect	Requests
Initial extraction	46.76	0	0	59
Simple repair	43.35	7	2	71
Indexed repair	44.84	5	12	69
Indexed re-extraction	44.12	8	32	71

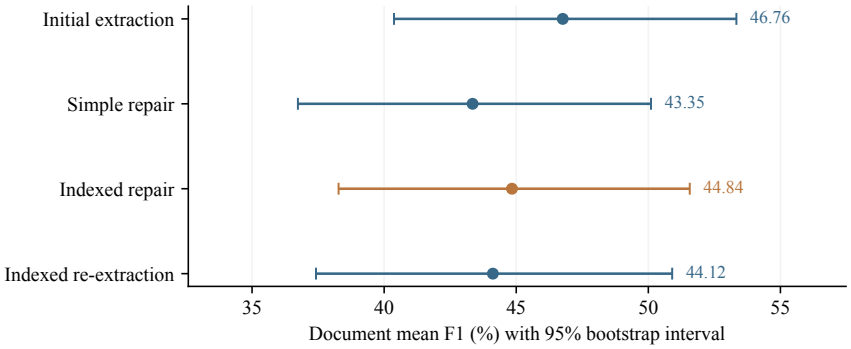


Fig. 9 Contract scores. Points show mean document F1. Bars show 95% CIs for each method. The text gives the paired repair–extraction CI.

Indexed repair leads indexed fresh extraction by 0.72 F1 points (95% paired CI: [-4.80,6.48]; sign-flip $p = 0.810$). The test gives 224 outputs from 270 requests, including retries. Failed outputs are scored as empty graphs. The score checks copied text against CUAD labels after whitespace cleanup.

The input graph has the highest F1 among these settings. Both repair methods lose correct input facts, and indexed repair also adds values that differ from the reference. Its small lead over indexed fresh extraction has an interval spanning both directions. The receipt gain therefore changes into a drop from the input on this contract task. Keeping correct fields and selecting the labelled text spans are the main repair targets shown by these results.

5.5 Human Review of Errors and Edits

Human review examines two decisions behind the automatic scores: whether a reference difference is an error in context, and whether an edit is acceptable in the output graph. The first concerns the input fact; the second also considers what the operation does to the graph.

Task D checks 200 reference differences sampled from the 302 triple differences in model-built inputs. Task E checks 200 actual edits sampled from 598 edits across five settings. Both use random sampling with seed 42. Each item is one difference or edit.

Table 10 Human scores. Agreement and κ use the two original 0/1/U answers. Positive counts use final labels. Acceptance rates use the cases with a 0 or 1 answer.

Task	Judgment	Agreement	κ	Positive / certain	U
D	Input error	80.0%	0.600	95/200 (47.5%)	0
D	Suggested edit acceptable	62.0%	0.334	86/200 (43.0%)	0
E	Input error	86.0%	0.677	134/200 (67.0%)	0
E	Actual edit acceptable	65.0%	0.128	134/199 (67.3%)	1

Table 11 Actual-edit acceptance by setting. The random sample has different group sizes. Rates use edits with a clear decision.

Configuration	Sampled	Acceptable	U
Archived Simple	105	63/104 (60.6%)	1
Archived Diagnosis + Gate	41	25/41 (61.0%)	0
Gemma Base + Gate	16	13/16 (81.3%)	0
Gemma Diagnosis + Gate	16	13/16 (81.3%)	0
Gemma SHACL Context + Gate	22	20/22 (90.9%)	0

A document can supply several items. Reviewers see the source, schema, input graph, and named operation. Task E also shows the full output graph. Method names and reference-quality hints are hidden.

Two reviewers with computer-science backgrounds and Chinese reading skills read the rules and practice examples. They then label the same 400 items on their own. For each item, they answer two questions: is the input wrong at this position (`is_error`), and is the add or remove operation acceptable (`repair_acceptable`)? They use 0, 1, or U and give reasons. U means the shown evidence is too limited for a decision. A third reviewer checks the source and graphs and resolves 154 items with a disagreement by choosing between the two answers. The 246 agreed items keep their labels. One edit-acceptance answer remains U. We retain the original labels and agreement scores.

Reviewers find input errors in 95/200 reference differences and accept 86/200 suggested edits. They accept 134/199 actual edits; one is uncertain. A missing value can be correct, yet adding it can leave two values in a field that allows one. The input-error and edit questions cover these two decisions. Original edit agreement in task E is 65.0% with $\kappa = 0.128$.

Table 11 reports the sample size and accepted edits for each setting. The combined acceptance rate pools these groups. The two human questions give different results because a correct value can still be added in a way that breaks a field rule. The original agreement scores show that reviewers differ more on edit acceptance than on identifying input errors. Together, the labels and reasons help locate problems in both field choice and edit execution.

Table 12 Research questions and test results.

Question	Comparison	Finding
RQ1	Added defects and model-built graphs	Diagnosis + Gate improves repair and F1. Direct copying matches all Chinese record references.
RQ2	Shared prompts and responses; source-line hints; edit selectors	Source and ID checks remove extra triples. The receipt index raises F1. Error reports lower mean F1 in matched tests. Equal selector weights score above learned weights on model-built inputs.
RQ3	Receipt and contract repair versus fresh extraction	The receipt test favours the existing graph after fence and ID cleanup. Indexed contract repair scores below the initial graph.

5.6 Discussion

Source format and field choice.

The Chinese records state field names and values directly, allowing Source-field Copy to match the references. Receipts require choosing among values in the document. The indexed prompt improves this choice in the new receipt test, and its gain is already present before filtering. The context tests show that field meanings and line hints work together. Contracts show a different result: repair loses correct input matches and ends below the initial graph. Source format and the ability to keep good input fields both shape the value of repair.

Checks and the meaning of a field.

The output gate removes triples that fail source and document-ID checks. Choosing the right source value for a relation is another part of repair. A receipt may contain both a subtotal and a final amount, each present in the text. The field meaning determines which one belongs under "total". Human review adds this context to exact reference matching and checks whether the resulting edit fits the graph's field rules.

Assessment and the next edit.

The assessment profile links detected problems to generation and action choice. The matched tests show lower mean F1 with the error report, while the selector logs show useful edits rejected by the acceptance score. These results identify two concrete design targets: the report should help the model choose field values, and the trial score should reward recovery of missing fields alongside quality and edit cost. Table 12 summarizes the answers to the three questions.

5.7 Code and Data

Code, prompts, data lists, model outputs, and scoring scripts are at https://github.com/turambar928/MCP_based_KG_construction. Each run saves model settings, parse status, requests, and token use. All responses are kept. Experiment READMEs give data sources, samples, checksums, and commands.

The human-review files include fixed task rules, the scorer, summary results, original agreement scores, and sample lists. Individual answers and reasons are kept private. Receipt scores use SROIE's human field labels.

6 Conclusion

We join graph checks with repair using source text and field constraints. An assessment profile records local, graph, and source errors. The main path cleans the input, asks a model for a complete graph, and filters its triples. A scale-aware selector also compares edits on graph copies, using quality gains, costs, and graph bounds. Shared prompts and scores before and after filtering show what each step changes.

Diagnosis + Gate repairs 98.00% of the added defects and raises F1 from 87.99% to 92.48% on model-built graphs. Field copying recovers every reference in the Chinese records. Source-line hints raise receipt F1 by 4.76 points over Simple. The full index and anchors alone have the same mean F1 in the later test. Indexed contract repair reaches 44.84% F1, compared with 46.76% for the input. Its difference from fresh extraction is 0.72 points (95% CI: $[-4.80, 6.48]$). Source and document-ID checks remove unwanted triples. Error reports lower mean F1 in the matched tests, and the default learned selector scores below equal weights on model-built inputs.

Human review finds input errors in 95/200 reference differences. Reviewers accept 134/199 edits across five settings, with one uncertain edit. These results give clear targets for better repair: select the right value for each field, keep correct input facts, and check the final graph.

Statements and Declarations

Data availability

Data lists, fixed sample IDs, model outputs, and scoring scripts are at https://github.com/turambar928/MCP_based_KG_construction. Experiment READMEs give the source datasets, access links, and data changes. We share human-review totals and the fixed sampling rules. Individual answers, written reasons, and source context are kept private.

Code availability

The same repository contains the code, full prompts, model settings, and commands for running the reported analyses.

References

Acosta M, Zaveri A, Simperl E, et al (2013) Crowdsourcing linked data quality assessment. In: International Semantic Web Conference, pp 260–276, https://doi.org/10.1007/978-3-642-41338-4_17

Anthropic (2025) Introducing Claude Haiku 4.5. Anthropic model release announcement, URL https://www.anthropic.com/news/claude-haiku-4-5, published 15 October 2025; accessed 17 September 2026

Auer S, Bizer C, Kobilarov G, et al (2007) DBpedia: A nucleus for a web of open data. In: The Semantic Web: 6th International Semantic Web Conference and 2nd Asian Semantic Web Conference (ISWC/ASWC 2007). Springer, pp 722–735, https://doi.org/10.1007/978-3-540-76298-0_52

Bian H (2025) LLM-empowered knowledge graph construction: A survey. arXiv preprint, https://doi.org/10.48550/arXiv.2510.20345, arXiv:2510.20345

Bollacker K, Evans C, Paritosh P, et al (2008) Freebase: A collaboratively created graph database for structuring human knowledge. In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, pp 1247–1250, https://doi.org/10.1145/1376616.1376746

Bordes A, Usunier N, Garcia-Duran A, et al (2013) Translating embeddings for modeling multi-relational data. In: Advances in Neural Information Processing Systems, URL https://papers.nips.cc/paper_files/paper/2013/hash/1cecc7a77928ca8133fa24680a88d2f9-Abstract.html

Chen M, Zhang W, Zhang W, et al (2019) Meta relational learning for few-shot link prediction in knowledge graphs. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 4217–4226, https://doi.org/10.18653/v1/D19-1431

Das R, Dhuliawala S, Zaheer M, et al (2018) Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning. In: International Conference on Learning Representations, URL https://arxiv.org/abs/1711.05851

Dettmers T, Minervini P, Stenetorp P, et al (2018) Convolutional 2D knowledge graph embeddings. In: Proceedings of the AAAI Conference on Artificial Intelligence, https://doi.org/10.1609/aaai.v32i1.11573

Ding B, Wang Q, Wang B, et al (2018) Improving knowledge graph embedding using simple constraints. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pp 110–121, https://doi.org/10.18653/v1/p18-1011

Dong X, Gabrilovich E, Heitz G, et al (2014) Knowledge vault: A web-scale approach to probabilistic knowledge fusion. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 601–610, https://doi.org/10.1145/2623330.2623623

Galárraga LA, Teflioudi C, Hose K, et al (2013) AMIE: Association rule mining under incomplete evidence in ontological knowledge bases. In: Proceedings of the 22nd International Conference on World Wide Web, pp 413–422, https://doi.org/10.1145/2488388.2488425

Gemma Team (2026) Gemma 4 technical report. arXiv preprint, https://doi.org/10.48550/arXiv.2607.02770, arXiv:2607.02770

Guo S, Wang Q, Wang L, et al (2016) Jointly embedding knowledge graphs and logical rules. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp 192–202, https://doi.org/10.18653/v1/d16-1019

Hendrycks D, Burns C, Chen A, et al (2021) CUAD: An expert-annotated NLP dataset for legal contract review. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/6ea9ab1baa0efb9e19094440c317e21b-Abstract-round1.html

Huang Z, Chen K, He J, et al (2019) ICDAR2019 competition on scanned receipt OCR and information extraction. In: 2019 International Conference on Document Analysis and Recognition (ICDAR). IEEE, pp 1516–1520, https://doi.org/10.1109/ICDAR.2019.00244

Ji S, Pan S, Cambria E, et al (2022) A survey on knowledge graphs: Representation, acquisition, and applications. IEEE Transactions on Neural Networks and Learning Systems 33(2):494–514. https://doi.org/10.1109/tnnls.2021.3070843

Knublauch H, Kontokostas D (2017) Shapes constraint language (SHACL). W3C Recommendation, URL https://www.w3.org/TR/2017/REC-shacl-20170720/

Kontokostas D, Westphal P, Auer S, et al (2014) Test-driven evaluation of linked data quality. In: Proceedings of the 23rd International Conference on World Wide Web, pp 747–758, https://doi.org/10.1145/2566486.2568002

Lao N, Mitchell T, Cohen WW (2011) Random walk inference and learning in a large scale knowledge base. In: Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, pp 529–539, URL https://aclanthology.org/D11-1049/

Lewis P, Perez E, Piktus A, et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, pp 9459–9474, URL https://papers.nips.cc/paper_files/paper/2020/hash/

6b493230205f780e1bc26945df7481e5-Abstract.html

Lin TW, Fierro G, Li H, et al (2025) Systematic evaluation of knowledge graph repair with large language models. arXiv preprint, https://doi.org/10.48550/arXiv.2507.22419, arXiv:2507.22419

Neo4j Labs (2024) LLM Knowledge Graph Builder. Open-source software repository, URL https://github.com/neo4j-labs/llm-graph-builder, version inspected at commit 5ff7af3e9bb9226e1bbecd02f70f8d98697727a7; accessed 16 September 2026

Nickel M, Murphy K, Tresp V, et al (2016) A review of relational machine learning for knowledge graphs. Proceedings of the IEEE 104(1):11–33. https://doi.org/10.1109/jproc.2015.2483592

Ortona S, Meduri VV, Papotti P (2018) Robust discovery of positive and negative rules in knowledge bases. In: Proceedings of the 34th IEEE International Conference on Data Engineering, pp 1168–1179, https://doi.org/10.1109/icde.2018.00108

Pan JZ, Razniewski S, Kalo JC, et al (2023) Large language models and knowledge graphs: Opportunities and challenges. arXiv preprint, https://doi.org/10.48550/arXiv.2308.06374, arXiv:2308.06374

Pan S, Luo L, Wang Y, et al (2024) Unifying large language models and knowledge graphs: A roadmap. IEEE Transactions on Knowledge and Data Engineering 36(7):3580–3599. https://doi.org/10.1109/tkde.2024.3352100

Paulheim H (2017) Knowledge graph refinement: A survey of approaches and evaluation methods. Semantic Web 8(3):489–508. https://doi.org/10.3233/sw-160218

Paulheim H, Bizer C (2014) Improving the quality of linked data using statistical distributions. International Journal on Semantic Web and Information Systems 10(2):63–86. https://doi.org/10.4018/ijswis.2014040104

Qu M, Tang J (2019) Probabilistic logic neural networks for reasoning. In: Advances in Neural Information Processing Systems, URL https://papers.nips.cc/paper_files/paper/2019/hash/13e5ebb0fa112fe1b31a1067962d74a7-Abstract.html

Qu M, Chen J, Xhonneux LP, et al (2021) RNNLogic: Learning logic rules for reasoning on knowledge graphs. In: International Conference on Learning Representations, URL https://arxiv.org/abs/2010.04029

Schlichtkrull M, Kipf TN, Bloem P, et al (2018) Modeling relational data with graph convolutional networks. In: European Semantic Web Conference, pp 593–607, https://doi.org/10.1007/978-3-319-93417-4_38

Shi B, Weninger T (2017) ProjE: Embedding projection for knowledge graph completion. In: Proceedings of the AAAI Conference on Artificial Intelligence, https:

//doi.org/10.1609/aaai.v31i1.10677

Suchanek FM, Kasneci G, Weikum G (2007) YAGO: A core of semantic knowledge. In: Proceedings of the 16th International Conference on World Wide Web, pp 697–706, https://doi.org/10.1145/1242572.1242667

Wang Q, Mao Z, Wang B, et al (2017) Knowledge graph embedding: A survey of approaches and applications. IEEE Transactions on Knowledge and Data Engineering 29(12):2724–2743. https://doi.org/10.1109/tkde.2017.2754499

Wang X, Chen L, Ban T, et al (2021) Knowledge graph quality control: A survey. Fundamental Research 1(5):607–626. https://doi.org/10.1016/j.fmre.2021.09.003

Wei J, Wang X, Schuurmans D, et al (2022) Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems, pp 24824–24837, https://doi.org/10.52202/068431-1800

White J, Fu Q, Hays S, et al (2023) A prompt pattern catalog to enhance prompt engineering with ChatGPT. In: Proceedings of the 30th Conference on Pattern Languages of Programs. ACM, https://doi.org/10.64346/PLoP2023p05

Wienand D, Paulheim H (2014) Detecting incorrect numerical data in DBpedia. In: European Semantic Web Conference, pp 504–518, https://doi.org/10.1007/978-3-319-07443-6_34

Xu L, Hu H, Zhang X, et al (2020) CLUE: A Chinese language understanding evaluation benchmark. In: Proceedings of the 28th International Conference on Computational Linguistics. International Committee on Computational Linguistics, pp 4762–4772, https://doi.org/10.18653/v1/2020.coling-main.419, URL https://aclanthology.org/2020.coling-main.419/

Yang B, Yih Wt, He X, et al (2015) Embedding entities and relations for learning and inference in knowledge bases. In: International Conference on Learning Representations, URL https://arxiv.org/abs/1412.6575

Yao S, Zhao J, Yu D, et al (2023) ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations, URL https://arxiv.org/abs/2210.03629

Zaveri A, Rula A, Maurino A, et al (2016) Quality assessment for linked data: A survey. Semantic Web 7(1):63–93. https://doi.org/10.3233/sw-150175

Zhu Z, Zhang Z, Xhonneux LP, et al (2021) Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In: Advances in Neural Information Processing Systems, pp 29476–29490, URL https://papers.nips.cc/paper_files/paper/2021/hash/f6a673f09493afcd8b129a0bcf1cd5bc-Abstract.html

Table 13 Four settings on added-defect inputs. Direct LLM uses a different prompt.

Variant	Repair	Over-repair	F1	Exact
No LLM completion	0.3200	0.0000	0.8472	0.0622
Direct LLM (separate prompt)	0.9244	0.0758	0.9520	0.8267
No constraint gate	0.9800	0.0227	0.9895	0.9333
Diagnosis + Gate	**0.9800**	**0.0167**	**0.9917**	**0.9333**

A Extra Tests

This section gives earlier method results, full filter and selector runs, the first receipt test, parsing details, and extra checks. Section 5 gives the main results.

A.1 Earlier Method and Filter Tests

Table 13 compares four saved settings. The no-completion setting returns the cleaned graph. Direct LLM uses its own prompt and skips cleanup. The filter test scores the same proposal before and after checks. Section 5.3 tests prompt text. Section A.2 tests each filter and selector step.

Model completion recovers most missing fields. Diagnosis + Gate reaches 99.17% F1, compared with 95.20% for Direct LLM. Filtering keeps the same repair rate and reduces over-repair by 0.60 points. It removes eight triples that fail source matching. All eight differ from the reference.

A.2 Tests of Filter and Selector Steps

We use 360 Gemma responses from the shared-prompt test and 180 responses from the new receipt test. We score each under eight filter settings: all checks, each of five checks removed, duplicate and field-count checks removed together, and all checks removed. The 540 responses give 4,320 scored outputs. Candidate order stays fixed. Removing two checks together tests how they affect each other.

Table 14 gives full-filter F1 minus each changed-filter F1. We first average changes across the three prompts for each document, then across documents. The full filter rejects 28 triples. Eleven have a wrong document ID and 17 fail source matching. All 28 differ from the reference. Other checks remove zero extra candidates. On model-built inputs, source matching adds 0.505 F1 points and the ID check adds 0.085 points. The 21 adjusted test p values exceed 0.05. The logs also list correct facts lost when a check is removed.

Current edit-selector tests.

We run the v2 selector on saved proposals for 225 added-defect and 225 model-built inputs. Thirteen settings give 5,850 outcomes with the default absolute prior score. We repeat them with a score relative to equal weights, giving another 5,850 outcomes. Both runs use `public-profile-v2` features and the retrained router. The archive keeps the older v1 runs.

Table 14 F1 change from each filter check, in percentage points. Each input group has 60 documents and three prompts. Positive values favour the full filter.

Removed check(s)	Controlled	Natural	Receipts
Duplicate	+0.000	+0.000	+0.000
Document head	+0.037	+0.085	+0.000
Relation type	+0.000	+0.000	+0.000
Source support	+0.037	+0.505	+0.060
Field count	+0.000	+0.000	+0.000
Duplicate + field count	+0.000	+0.000	+0.000
All checks	+0.073	+0.590	+0.060

Table 15 V2 selector with the default absolute prior score. Accepted counts are actions or bundles, shown as added-defect / model-built. Each F1 uses 225 documents.

Setting	Controlled F1	Natural F1	Accepted
Always-on uniform	0.8472	0.8833	0 / 10
Learned prior, always on	0.8472	0.8799	0 / 10
Learned prior and trigger	0.8472	0.8799	0 / 10
No prior penalty	0.8472	0.8833	0 / 10
No action cost	0.8930	0.8813	68 / 13
No density bound	0.8472	0.8833	0 / 10
No quality bounds	0.8472	0.8833	0 / 10
One iteration	0.8472	0.8833	0 / 10
No rule candidates	0.8472	0.8833	0 / 9
Rule candidates only	0.8472	0.8799	0 / 10
No local score/bounds	0.8472	0.8799	0 / 0
No graph score/bound	0.8472	0.8833	0 / 10
No source score/bound	0.8472	0.8799	0 / 10

The base setting uses equal weights and keeps the selector on. Each test changes one named step. The trigger test starts from the always-on learned-weight setting. Removing a score sets its term to zero and removes its bounds. Other weights stay the same. Rule detectors and violation rewards stay active. Removing rule candidates drops detector actions and direct source-field additions. The graph profile and saved model proposals stay in use.

The default equal-weight selector reaches 84.72% F1 on added-defect inputs and accepts zero edits. Removing edit cost raises F1 to 89.30% with 68 accepted actions. On model-built inputs, equal weights reach 88.33% F1. Learned weights and the learned trigger reach 87.99%. Every setting keeps all initially correct reference facts in both groups. The released files give document scores, trial edits, rejection reasons, and stop reasons. Holm correction groups the 24 component tests for each prior score form.

Table 16 V2 selector on saved proposals. Relative scores use equal weights as the zero point. Accepted counts are added-defect / model-built.

Prior / offset	Controlled F1	Natural F1	Accepted
Uniform / absolute	0.8472	0.8833	0 / 10
Uniform / relative	0.8472	0.8833	0 / 10
Learned / absolute	0.8472	0.8799	0 / 10
Learned / relative	0.8929	0.8764	156 / 51

Router training and prior scores.

The v2 router has an $8 \to 32 \to 16$ network, one repair output, and three edit-type outputs (884 parameters). The feature scaler uses 2,098 training rows. Validation and test each have 450 rows grouped by document. Repair and edit-type classification on added-defect inputs both reach 100%. Training labels follow the known added defects.

Table 16 gives four weight/score settings from the earlier v2 test. The full runs above include these settings. With learned weights, the relative score raises added-defect F1 from 84.72% to 89.29%. Model-built F1 falls from 87.99% to 87.64%. Both equal-weight settings return the same graphs. The absolute score remains the default.

A.3 Receipt Setup and First Test

We use SROIE receipt text and human field labels (Huang et al. 2019). A random sample selects 60 of 626 documents from the corrected trainval mirror (seed 20260922). We join text lines in their stored order. The model receives this text and four field names: company, date, address, and total. The scorer reads the field labels. One receipt has no address label, leaving 239 scored fields. Every method is scored on these fields. The model receives all four field names.

Gemma first extracts a graph. Three repair prompts receive this same graph with the settings in Section 4.4. We fix samples and settings before calling the model. The test has 60 extraction calls and 180 repair outputs. Each repair response is scored before and after filtering. We also run source copying and rule repair with their original settings. The main score uses exact triples. A second score ignores case and whitespace in values.

The input reaches 79.44% F1. After filtering, base and error-report prompts both reach 79.15%. SHACL context reaches 79.56%. The error-report minus base difference is 0.00 points (95% CI: $[0.00, 0.00]$; paired randomization $p = 1.0000$). Field copying reaches 0.00% F1 on receipts and 100% on the named-field Chinese records.

Of 239 reference values, 220 occur exactly in the source text. The other 19 differ from the source strings. Filtering rejects 3 candidates, all matching reference values. They use the same address in three settings. The source has a double space; the parser replaces it with a single space in the candidate. After the run, we apply the same whitespace rule to the source. All three settings then keep the address. F1 is 79.44% for base and error-report prompts and 79.86% for SHACL. Exact graph match stays at 33.33%. Table 17 gives the original-rule scores.

Table 17 First SROIE test on 60 receipts. Exact match uses labelled fields. Normalized F1 ignores case and whitespace in values.

Method	Triple F1	Exact match	Normalized F1	Facts kept
Extracted input	0.7944	0.3333	0.8153	1.0000
Rule Only	0.7944	0.3333	0.8153	1.0000
Source-field Copy	0.0000	0.0000	0.0000	0.0000
Base, raw	0.7944	0.3333	0.8153	1.0000
Base, gate	0.7915	0.3333	0.8123	0.9944
Diagnostic, raw	0.7944	0.3333	0.8153	1.0000
Diagnostic, gate	0.7915	0.3333	0.8123	0.9944
SHACL context, raw	0.7986	0.3333	0.8194	1.0000
SHACL context, gate	0.7956	0.3333	0.8165	0.9944

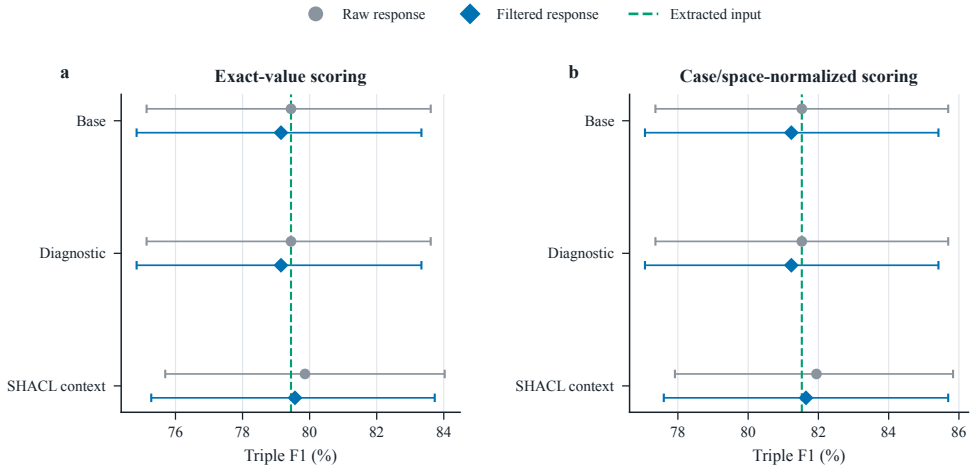


Fig. 10 First receipt scores. Points show mean F1 and 95% document-bootstrap CIs. Each raw/gated pair uses one response. The dashed line shows input F1.

The base and error-report prompts keep all input graphs unchanged before filtering. Of 49 reference differences, 27 involve case, spaces, punctuation, or amount format. These findings guide the field meanings, source-line hints, and shared whitespace rule used in the new receipt test.

The run has 240 recorded requests, including retries, and 240 parsed outputs. Mean repair request times are 7.87, 7.92, and 8.05 seconds for base, error-report, and SHACL prompts. Mean input lengths are 661, 715, and 2276 tokens. Request times include pacing. Extraction and context setup are counted apart.

New receipt development and requests.

The first 60 receipts, the 20 development receipts, and the 60 new-test receipts have separate IDs and source texts after whitespace cleanup. Sampling uses seed 20260923. The index design is set on the development set, and test labels are downloaded after predictions are saved.

Table 18 Extra scores after collection on sixty receipts. Fence-compatible removes outer Markdown fences. ID-normalized also assigns the known document ID before filtering. Both use the saved responses.

Method	Fence-compatible F1	ID-normalized F1	Wrong-ID graphs
Fresh extraction	0.6333	0.7256	18/60
Indexed extraction	0.6105	0.7486	28/60
Repair	0.7917	0.7917	0/60
Indexed repair	0.8393	0.8393	0/60

Development F1 is 80.00% for the input, 82.50% for Simple, 85.00% for Evidence Index, and 80.00% for SHACL. We keep one index design and all three methods for the test. The main comparison is Evidence Index minus Simple in mean document F1. The code and scoring rules are fixed before testing.

Simple, Evidence Index, and SHACL change 31, 39, and 28 input graphs. Their mean repair request times are 7.91, 7.88, and 8.11 seconds. Mean input lengths are 1784, 2153, and 3404 tokens. The 240 requests produce 60 extraction and 180 repair outputs. Request times include pacing and retries. Each raw/gated score pair uses one response.

A.4 Receipt Repair and Fresh-Extraction Details

We collect 240 further Gemma responses on the same sixty new-test receipts. The settings are repair or fresh extraction, with or without the index. All share source lines, field meanings, document ID, and output limit. Repair also receives the existing graph. The main score difference is indexed repair minus indexed fresh extraction after filtering. All 240 responses have JSON inside Markdown fences. The fixed strict parser marks them as failures and scores every setting at zero F1.

After collection, we remove each outer fence and apply the same schema checks. All 240 responses then parse. Table 18 gives these extra scores. Indexed repair reaches 83.93% F1 and indexed fresh extraction reaches 61.05%. The paired difference is 22.88 points (95% CI: 14.84 to 31.35).

Indexed fresh extraction gives the wrong document ID in 28 graphs. Indexed repair uses the right ID in all sixty. Assigning the known ID to all four settings raises indexed fresh-extraction F1 to 74.86%. Indexed repair leads by 9.07 points (95% CI: 5.22 to 13.10). This ID change keeps field values fixed. The original 22.88-point gap includes both ID and field-value effects. Initial graph creation is counted apart from these repair calls.

Fields found by the line index.

We locate the original follow-up's 240 reference fields in the source lines. Matching keeps case and punctuation and cleans whitespace. For a value spread over several lines, the index must include all lines of one occurrence. Of the 240 values, 188 occur in index lines or nearby lines, 36 occur outside them, and 16 differ from the source text. Among the 188 covered values, Simple returns 163 exactly and Evidence Index

Table 19 Earlier selector results on 225 inputs per group. The oracle uses references to find the best saved subset. One document can have several trials.

Diagnostic	Controlled	Natural
Preprocessed graph F1	0.8472	0.8799
Uniform optimizer F1	0.8472	0.8833
Fixed-proposal oracle F1	0.9964	0.9173
Initially no detected violation	86	198
Imperfect among those graphs	72	50
Improvement available among those graphs	70	8
Improving trials with non-positive utility	212	1

returns 174. Both return 26 of the 36 outside values. They return zero of the sixteen values that differ from the source. The index's net gain of eleven exact fields comes from the covered group.

A.5 Calling Repair after Visible Errors

We test a call rule based on visible ID, relation, duplicate, field-count, and source-match errors. It selects either the input or the saved repair output for each document. On the 450-item model-built/clean stream, it calls repair on 6.22% of inputs. It misses 63.64% of graphs with reference differences. Its F1 is 94.46%, compared with 95.82% for always calling repair. The visible checks miss many missing or wrongly assigned field values. The main method calls repair on every document.

Useful edits in the earlier selector.

We use saved proposals and trial graphs from the earlier selector. These runs came before the shared-feature and candidate-start changes in Table 16. For each document, we try every subset of model edit bundles and score its graph against the reference. We apply the saved edits with stopping, cost, and graph-bound checks turned off. We call the best reference-scored subset the oracle. This analysis uses model bundles and reference values.

We try 2,144 subsets across 450 inputs. Table 19 lists the results. On added-defect inputs, 86 graphs have zero detected violations. Of these, 72 miss reference fields and 70 have a proposal subset that raises F1. Among the checked trials, 212 raise reference F1 but have scores of zero or less. The matching model-built counts are 8 graphs and 1 trial.

Removing edit cost raises F1 on 67 added-defect documents, keeps it the same on 158, and lowers it on 0. On model-built inputs, it raises F1 on 0, keeps it the same on 222, and lowers it on 3. The three graphs with lower F1 already differ from the reference after cleanup. All 148 exact model-built inputs keep their F1. The paired added-defect gain is 4.58 points (95% document-bootstrap CI: 3.69–5.54). The same cost change has different effects on the two input groups.

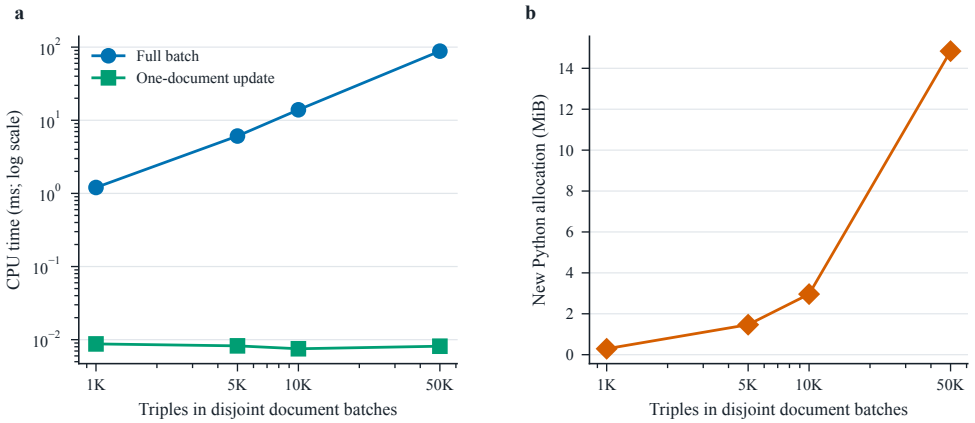


Fig. 11 Time and memory for reports on separate document batches. Local updates check one changed document. Memory measures new batch storage with shared source text.

A.6 Time and Memory for Graph Checks

We time the report in Eq. (2) on separate document batches with 1K–50K triples. A full pass checks every document. A local update checks one changed document. Full times are medians of seven runs. Local times are medians of 200 runs. Table 21 gives the values. Memory measures the peak new Python memory for building the batch. Source text and schema objects are shared. Model-call times are recorded apart.

A.7 Extra Quality and Extraction Checks

A Gemma judge scores 180 source–triple pairs with method names hidden. Its source score has Pearson $r = 0.794$ and Spearman $\rho = 0.807$ with exact reference validity (both $p < 2.4 \times 10^{-40}$). A fixed 50-item sample has the same mean, 0.96, in five further runs.

We compare our extractor with the `LLMGraphTransformer` core used by Neo4j Labs' Graph Builder (Neo4j Labs 2024) on 45 balanced TNEWS documents from CLUE (Xu et al. 2020). Category accuracy is 0.556 for our field-based extractor and 0.533 for Graph Builder. The paired test gives $p = 1.00$.

A.8 Request Costs and Check Times

Table 20 Request costs. Mean times include retries and pacing. Source setup and queue time are counted apart.

Input	Context	Calls	Input tokens	Output tokens	Wall (s)
Controlled	Base	1.00	947	517	8.58
Controlled	Diagnostic	1.00	1017	512	8.61
Controlled	SHACL	1.00	3080	513	8.45
Natural	Base	1.00	832	458	7.51
Natural	Diagnostic	1.00	901	462	7.90
Natural	SHACL	1.00	2822	455	8.16

Table 21 Time and memory for graph reports. Memory counts peak new batch storage in MiB. Source texts are shared.

Triples	Documents	Full (ms)	Local (ms)	Memory (MiB)
1,000	143	1.20	0.009	0.29
5,000	716	6.08	0.008	1.47
10,000	1,434	13.90	0.008	2.96
50,000	7,173	88.10	0.008	14.84